

PEC 3

Visión Artificial con Deep Learning

Autor: Jaime Ciriaco Beaumont

Asignatura: Visión Artificial - Máster Universitario en Investigación en Inteligencia Artificial - UNED

Curso: 2025/2026

1. Introducción

En esta PEC se han trabajado varias tareas básicas de visión artificial con aprendizaje profundo: localización, clasificación y segmentación. La primera parte se ha centrado en la figura compleja de Rey, a partir de imágenes escaneadas de dibujos realizados a mano. En este caso se han resuelto dos problemas: localizar automáticamente la región de interés de la figura dentro de la imagen original y estimar su orientación para aproximarla a la posición del patrón. Todo ello permite trabajar dos tareas básicas de visión artificial: la localización de una región de interés y la clasificación de la orientación de una imagen.

La segunda parte se ha centrado en la segmentación semántica de fotografías de cavidad oral. Este bloque ha requerido un trabajo previo de preparación del dataset, ya que no se partía directamente de un conjunto final de imágenes y máscaras listo para entrenar el modelo, sino que para ello, se ha combinado SMART-OM con anotaciones colaborativas de imgsMIIA compartidas en el foro de Teams, obteniendo finalmente un dataset binario de segmentación de lengua frente a fondo.

La metodología seguida ha sido común en ambos bloques de la PEC3: primero se revisan y limpian los datos, después se formula el problema de aprendizaje, se entrena una solución propia con Keras/TensorFlow en local y, finalmente, se compara con una alternativa basada en YOLO. Esta comparación no pretende hacer equivalentes ambas aproximaciones, ya que YOLO parte de pesos preentrenados, pero sí permite valorar si los resultados obtenidos por las redes propias son razonables frente a una arquitectura más optimizada.

2. Localización y orientación de la figura de Rey

2.1. Análisis del problema

El primer bloque trabaja con imágenes escaneadas de dibujos de la figura compleja de Rey. A diferencia de otros problemas habituales de visión artificial, las imágenes no contienen objetos naturales con texturas, color o fondos claramente diferenciados, sino dibujos lineales hechos a mano. Por tanto, la información principal está en la forma global, la composición y la posición del dibujo dentro de la hoja.

Se han planteado dos tareas supervisadas:

Tarea	Entrada	Salida	Tipo de problema
Localización	Imagen escaneada completa	Bounding box de la figura	Regresión / detección
Orientación	ROI de la figura	Clase de rotación	Clasificación

Para la localización se usa como referencia la bounding box incluida en traza_REY.csv. Para la orientación se usa la clase de rotación manual, con cuatro valores posibles: 0, 90, 180 y -90.

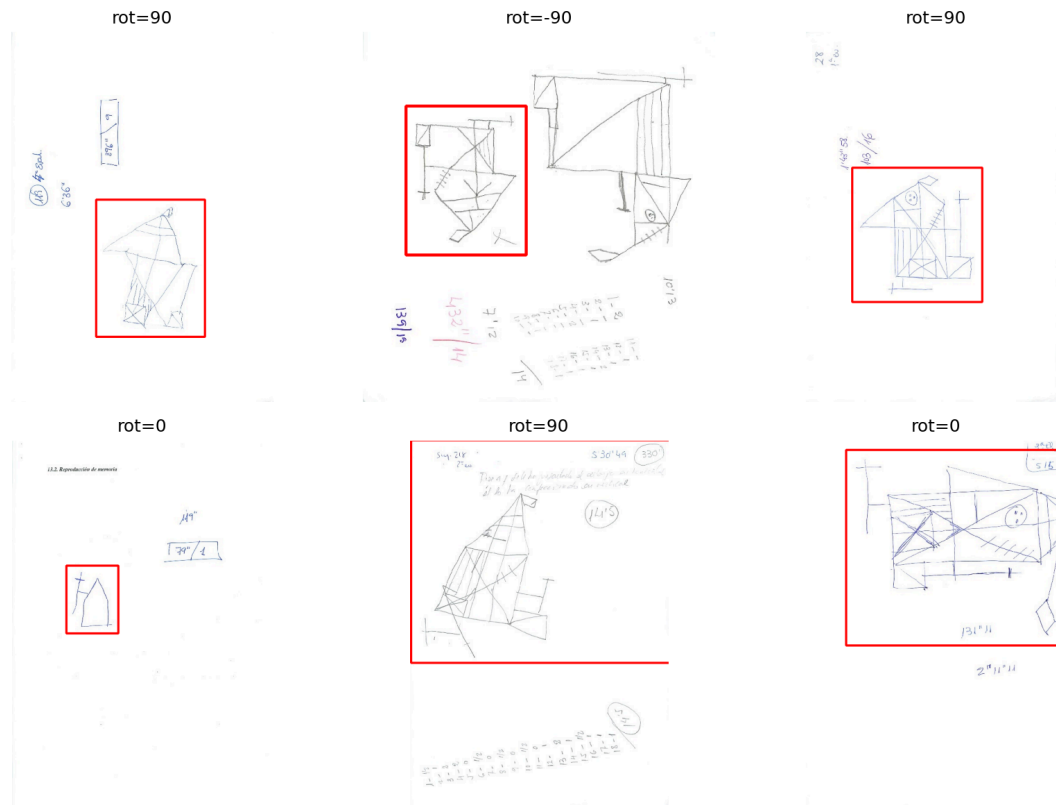


Figura 1. Ejemplos del dataset de la figura de Rey con la bounding box anotada. Esta visualización se utilizó para comprobar que las cajas extraídas de traza_REY.csv estaban bien interpretadas antes de entrenar los modelos.

2.2. Preparación de datos

El fichero traza_REY.csv no se leía como un CSV estándar, ya que cada línea contenía campos de texto con estructura id, fname, forig, BB y rot. Por ello, el primer paso fue parsear el fichero y generar una tabla limpia con el nombre de la imagen original, la ROI, la bounding box y la rotación correspondiente.

Después de la limpieza se visualizaron varias imágenes con sus cajas anotadas. Este paso fue necesario porque las anotaciones proceden de un proceso manual y podían existir errores. En aprendizaje supervisado, la calidad del ground truth es especialmente importante: si la anotación está mal, el modelo aprende una referencia incorrecta.

2.3. Localización con CNN propia

La primera solución de localización se implementó mediante una CNN sencilla en Keras. El modelo recibe la imagen completa redimensionada y devuelve cuatro valores normalizados correspondientes a la bounding box. Esta salida se interpreta como una regresión, ya que no se predice una categoría, sino una posición y un tamaño.

La métrica principal utilizada fue la IoU, porque mide el grado de solapamiento entre la caja predicha y la caja real.

El resultado obtenido fue:

Modelo	Tarea	Métrica	Resultado
CNN Keras	Localización ROI	IoU media	0.7286

El resultado es razonable para una red propia sencilla. Visualmente, el modelo localiza la figura en la mayoría de los casos, aunque presenta errores cuando el dibujo es pequeño, aparece desplazado o tiene poco contraste.

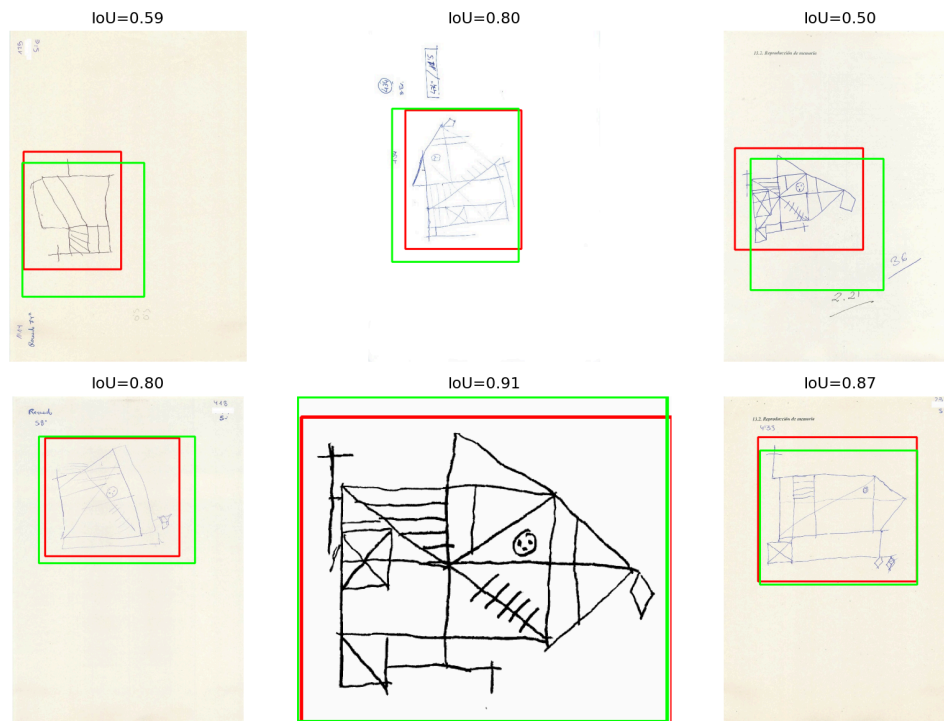


Figura 2. Predicciones de bounding box obtenidas con la CNN propia sobre imágenes de test. Se comparan las cajas reales con las predichas para valorar visualmente el error de localización.

2.4. Localización con YOLO

Como comparación, se preparó el mismo problema en formato YOLO detection. Las cajas del CSV se transformaron al formato requerido por YOLO, usando centro, ancho y alto normalizados. Solo se definió una clase: figura de Rey.

YOLO obtuvo un rendimiento superior al de la CNN propia:

Modelo	Tarea	Métrica	Resultado
CNN Keras	Localización ROI	IoU media	0.7286
YOLO11n detect	Localización ROI	mAP50-95	0.925

No son métricas idénticas, por lo que no deben interpretarse como una comparación directa uno a uno. Aun así, el resultado muestra que YOLO localiza de forma más robusta la zona de interés, algo esperable al tratarse de una arquitectura diseñada específicamente para detección y con pesos preentrenados.

2.5. Orientación como problema de clasificación

Para la segunda tarea se eligió formular la orientación como clasificación. Esta decisión se tomó porque el dataset ya proporciona cuatro clases discretas de rotación, de modo que el problema se reduce a reconocer cuál de ellas aproxima mejor la orientación del patrón.

Se entrenó una CNN propia con cuatro clases: -90, 0, 90 y 180. La salida es una distribución de probabilidad sobre esas clases. La métrica principal fue la accuracy en test, acompañada de la matriz de confusión para estudiar los errores.

Modelo	Tarea	Accuracy test
CNN Keras	Clasificación de orientación	0.9248
YOLO11n-cls	Clasificación de orientación	0.9549

La CNN propia obtuvo una accuracy alta, por lo que la formulación como clasificación resulta adecuada. YOLO classification mejoró ligeramente el resultado, probablemente por su arquitectura y por el aprendizaje transferido. La clase 180 debe interpretarse con cautela, ya que el número de ejemplos disponibles era reducido.

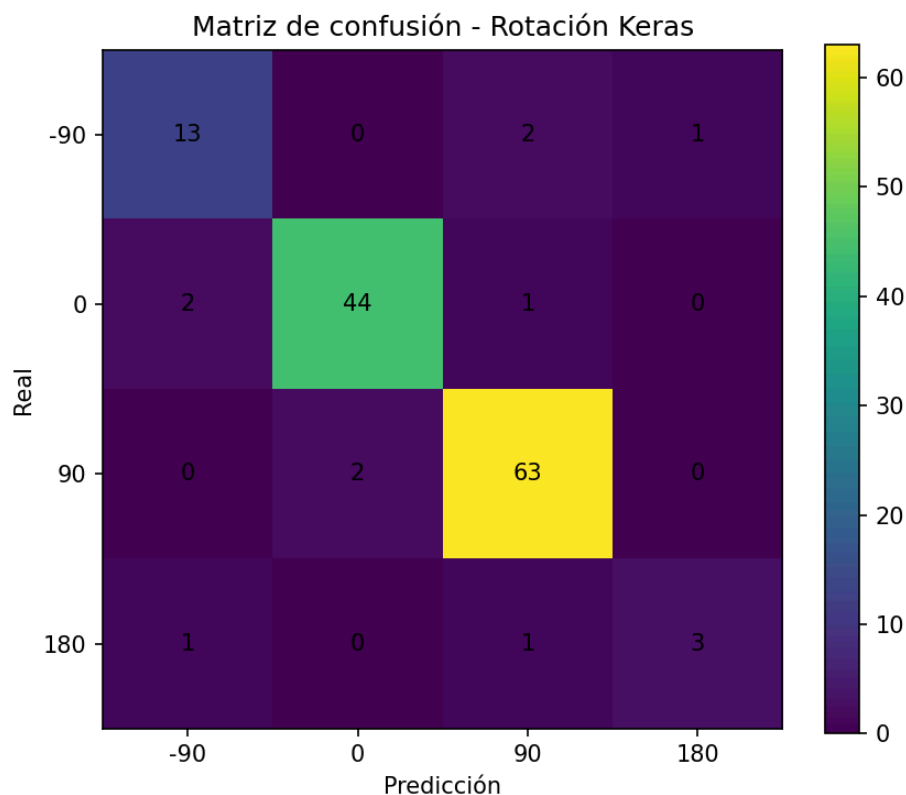


Figura 3. Matriz de confusión del modelo CNN para clasificación de orientación de la figura de Rey. Permite observar en qué clases se concentran los errores de rotación.

2.6. Conclusión del bloque Rey

En el bloque de Rey, los modelos propios permiten resolver las tareas de forma aceptable, pero YOLO mejora los resultados tanto en localización como en orientación. La limpieza inicial de traza_REY.csv y la visualización de anotaciones fueron pasos clave, ya que permitieron comprobar que las referencias manuales se estaban usando correctamente.

La conclusión principal es la prevista, que las redes propias son útiles para entender y resolver el problema, mientras que YOLO aporta una solución más robusta cuando los datos se adaptan a su formato.

3. Segmentación semántica de cavidad oral

3.1. Planteamiento

El segundo bloque aborda la segmentación semántica de fotografías de cavidad oral. El objetivo general es aportar información visual adicional a un sistema de análisis de imágenes orales. Debido a la limitación de tiempo y a la dificultad de anotar todas las estructuras de la boca, se decidió restringir el trabajo a una única estructura: la lengua.

La tarea se formuló como segmentación binaria:

Valor	Significado
0	Fondo
1	Lengua

Aunque durante la preparación se conservaron las etiquetas anatómicas DT, VT y LT, el modelo se entrenó de forma binaria, es decir, lengua frente a no lengua. Esta decisión simplifica el problema y permite aprovechar todos los ejemplos de lengua como clase positiva. La segmentación multiclase por zona anatómica se plantea como una posible mejora futura.

3.2. Fuentes de datos

Para construir el dataset final se utilizaron dos fuentes:

1. SMART-OM: como dataset base de imágenes de mucosa oral.
2. imgsMIIA colaborativo: formado por anotaciones compartidas en el foro.

El uso de un conjunto colaborativo está justificado por el tamaño reducido de los datasets médicos y por la necesidad de ampliar el número de imágenes anotadas. Mi aportación técnica en este bloque ha consistido en limpiar, transformar y unificar anotaciones dispersas para obtener un dataset final entrenable.

3.3. Preparación de SMART-OM

En SMART-OM se revisaron las anotaciones en formato VIA2. A partir de ellas se extrajeron regiones útiles y se filtraron las imágenes relacionadas con la lengua. En este conjunto aparecieron ejemplos de lengua dorsal y ventral, pero no de lengua lateral.

Clase	Muestras SMART-OM
DT	386
VT	367

Clase	Muestras SMART-OM
LT	0
Total	753

Estas anotaciones se transformaron a máscaras binarias y se adaptaron al mismo tamaño que el resto del dataset.

3.4. Preparación de imgsMIIA colaborativo

También se procesaron varios zips colaborativos descargados del foro. Se ignoraron duplicados y carpetas auxiliares como __MACOSX o .ipynb_checkpoints, ya que podían introducir ficheros repetidos o no válidos.

Las aportaciones utilizadas fueron:

- imgsMIIA_ffortes.zip
- imgsMIIA_mpozaz.zip
- imgsMIIA_msastre.zip
- imgsMIIA_Zarza.zip

A partir de los JSON se generaron máscaras binarias de lengua. El resultado final fue:

Clase	Muestras imgsMIIA
DT	397
VT	510
LT	644
Total	1551

En el caso de Zarza se detectó que existían ficheros DT, pero no contenían regiones útiles. Se comprobaron tanto los JSON de 04. Jsn files como los de Proyects, ambos con 166 entradas y 0 regiones válidas. Por ello, esas anotaciones no se incorporaron como DT.

3.5. Dataset final combinado

Una vez procesadas ambas fuentes, se creó un dataset final con imágenes y máscaras redimensionadas a 256x256. La distribución quedó así:

Origen	Muestras
SMART-OM	753
imgsMIIA_colab	1551
Total	2304

Por clase anatómica:

Clase	Total
DT	783
VT	877
LT	644
Total	2304

La partición final fue:

Split	Imágenes	Máscaras
Train	1612	1612
Validation	346	346
Test	346	346

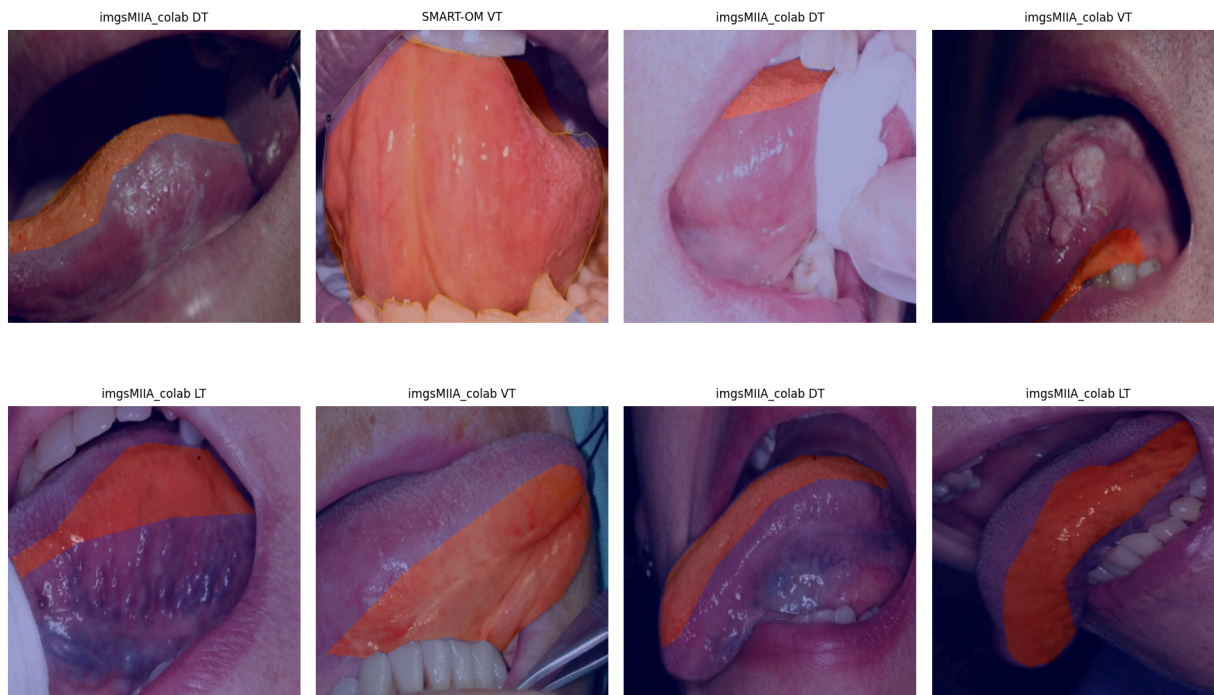


Figura 4. Muestras del dataset final combinado SMART-OM + imgsmiia con máscaras de lengua superpuestas. La figura resume la variabilidad visual del conjunto usado para entrenar los modelos de segmentación.

4. U-Net para segmentación binaria

4.1. Modelo y entrenamiento

Como modelo propio se implementó una U-Net ligera en Keras/TensorFlow. U-Net es adecuada para segmentación porque combina una parte de codificación, encargada de extraer características, con una parte de decodificación, encargada de recuperar la resolución espacial. Las conexiones entre encoder y decoder ayudan a conservar detalles de borde.

La entrada del modelo es una imagen RGB de 256x256x3, y la salida es una máscara 256x256x1 con activación sigmoide. La función de pérdida combina binary crossentropy y Dice loss, para equilibrar la clasificación píxel a píxel con una métrica sensible al solapamiento.

Parámetro	Valor
Tamaño de imagen	256x256
Batch size	4
Épocas	20

Parámetro	Valor
Optimizador	Adam
Learning rate	1e-4
Loss	BCE + Dice loss
Métricas	Dice, IoU, binary accuracy

Inicialmente se planteó una U-Net mayor, pero el entrenamiento se estaba ejecutando en CPU porque TensorFlow no detectaba la GPU. Por ello se redujo la arquitectura para completar el experimento en un tiempo razonable dentro del entorno disponible.

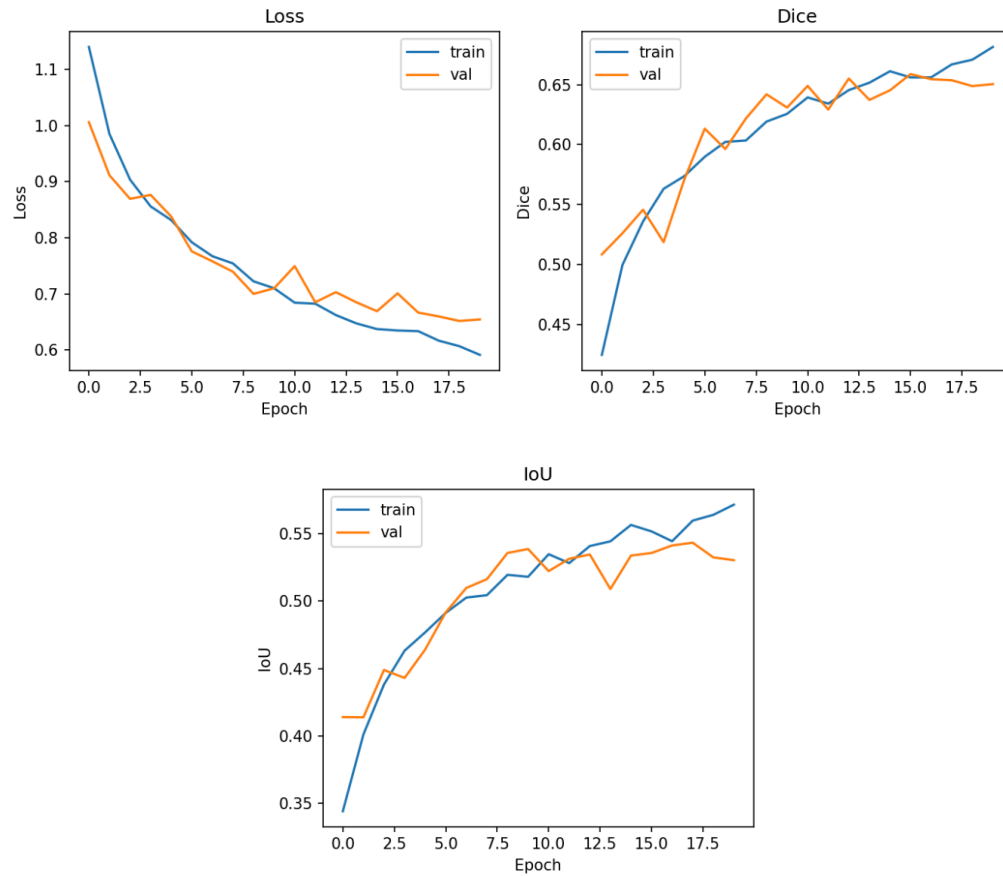


Figura 5. Curvas de entrenamiento de la U-Net ligera. Se muestran la evolución de la loss, Dice e IoU en entrenamiento y validación.

4.2. Resultados de U-Net

La evaluación directa de Keras sobre test produjo:

Modelo	Dice test	IoU test	Binary accuracy
U-Net light	0.6572	0.5236	0.8449

Además, para comparar de forma homogénea con YOLO, se calculó Dice e IoU manualmente por imagen sobre las muestras comunes de test. En esa evaluación común, la U-Net obtuvo:

Modelo	Dice medio	IoU medio
U-Net light	0.4014	0.3416

La diferencia entre la evaluación directa y la media manual se debe a que no agregan los resultados exactamente de la misma manera. Para la comparación final se usó la evaluación manual común, ya que se aplica de forma idéntica a ambos modelos.

Por origen, la U-Net obtuvo:

Origen	Dice	IoU
SMART-OM	0.9018	0.8385
imgsMIIA_colab	0.1609	0.1027

Este resultado indica que el modelo aprende muy bien SMART-OM, pero generaliza mal sobre imgsMIIA. La causa probable es una diferencia de dominio: iluminación, encuadre, estilo de anotación y variabilidad de las imágenes.

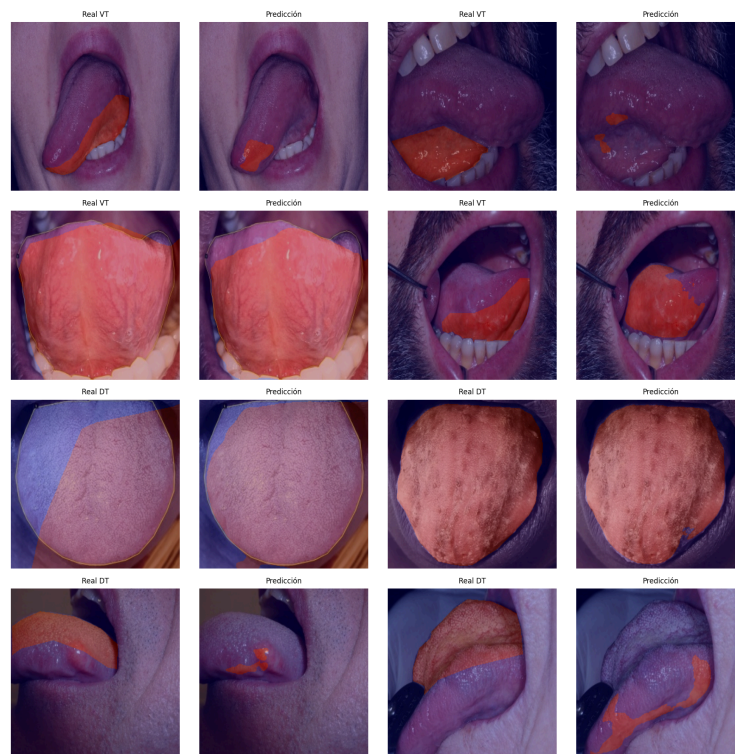


Figura 6. Ejemplos de predicción de la U-Net en test. En cada caso se compara la máscara real con la máscara predicha para analizar visualmente aciertos y errores.

5. YOLO segmentation

5.1. Preparación del dataset YOLO

Para comparar con una arquitectura preentrenada se preparó el dataset en formato YOLO segmentation. A diferencia de U-Net, YOLO no entrena directamente con máscaras PNG, sino con etiquetas .txt que contienen polígonos normalizados. Por ello, cada máscara binaria se convirtió a contorno usando OpenCV.

El dataset YOLO quedó con 2303 muestras, una menos que el dataset original porque una máscara no generó un polígono válido.

Split	Muestras YOLO
Train	1612
Validation	346
Test	345

5.2. Entrenamiento y validación

Se entrenó YOLO11n-seg durante 20 épocas, con tamaño de imagen 256 y batch size 4. Aunque el equipo dispone de una RTX 4070, el entorno de PyTorch instalado era CPU, por lo que el entrenamiento se ejecutó en CPU.

La validación interna de YOLO en test produjo las siguientes métricas para máscara:

Métrica YOLO mask	Valor
Precision	0.507
Recall	0.501
mAP50	0.575
mAP50-95	0.304

5.3. Evaluación manual corregida

Además de las métricas internas de YOLO, se calculó Dice e IoU manualmente para poder compararlo con U-Net. Durante la primera evaluación apareció un problema: algunos valores de Dice eran mayores que 1, lo cual es imposible. Al revisar las dimensiones se detectó que algunas máscaras reales se cargaban como (256, 256, 1), mientras que las predicciones estaban en (256, 256). Esto provocaba broadcasting incorrecto en NumPy.

La solución fue forzar todas las máscaras a 2D antes del cálculo. Tras esa corrección, los rangos fueron válidos:

Métrica	Mínimo	Máximo
Dice	0.0000	0.9865
IoU	0.0000	0.9734

La evaluación manual corregida de YOLO fue:

Modelo	Dice medio	IoU medio
YOLO11n-seg	0.4711	0.3977

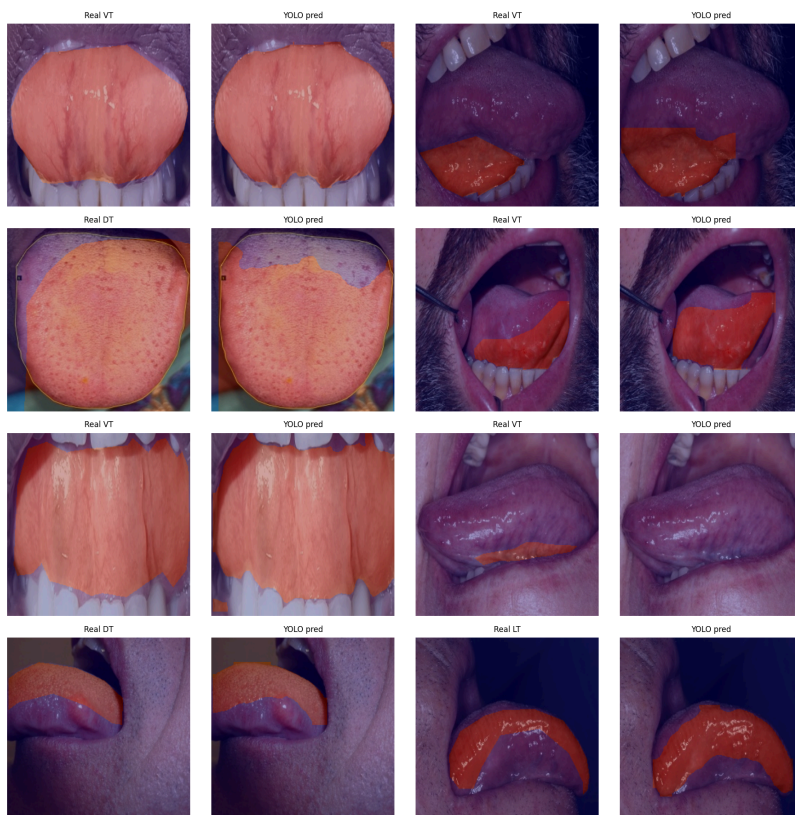


Figura 7. Ejemplos de predicción de YOLO11n-seg sobre el conjunto de test. La comparación con las máscaras reales permite observar los casos en los que YOLO ajusta bien la lengua y aquellos donde falla por iluminación, bordes poco claros u oclusiones.

6. Comparación final

La comparación final entre U-Net y YOLO se realizó sobre las 345 imágenes comunes de test.

Modelo	Dice medio	IoU medio
U-Net light	0.4014	0.3416
YOLO11n-seg	0.4711	0.3977

YOLO obtiene mejores métricas globales, aunque la diferencia no es enorme. La comparación por origen muestra un comportamiento más interesante:

Origen	Modelo	Dice	IoU
SMART-OM	U-Net light	0.9018	0.8385
SMART-OM	YOLO11n-seg	0.8664	0.7791
imgsMIIA_colab	U-Net light	0.1609	0.1027
imgsMIIA_colab	YOLO11n-seg	0.2811	0.2144

U-Net funciona ligeramente mejor sobre SMART-OM, mientras que YOLO generaliza mejor sobre imgsMIIA. Esto refuerza la idea de que el conjunto colaborativo es más heterogéneo y difícil.

Por clase anatómica:

Clase	Modelo	Dice	IoU
DT	U-Net light	0.5102	0.4694
DT	YOLO11n-seg	0.5502	0.4860
LT	U-Net light	0.1861	0.1194
LT	YOLO11n-seg	0.3995	0.3079
VT	U-Net light	0.4637	0.3919
VT	YOLO11n-seg	0.4536	0.3854

YOLO mejora especialmente en LT. En VT ambos modelos quedan muy parecidos, y en DT YOLO queda ligeramente por encima.

7. Análisis de errores

Los errores más habituales aparecen en imágenes con sombras, iluminación irregular, oclusiones, dientes o guantes muy próximos a la lengua, bordes poco definidos y diferencias claras de estilo entre datasets. También influye la forma de anotar: algunas máscaras siguen la lengua con precisión, mientras que otras son más amplias o más irregulares.

La diferencia entre SMART-OM e imgsMIIA es el resultado más importante del bloque de segmentación. SMART-OM parece más homogéneo y por eso la U-Net lo aprende muy bien. En cambio, imgsMIIA reúne imágenes y anotaciones de procedencias diferentes, lo que dificulta la generalización. YOLO, al partir de pesos preentrenados, parece resistir algo mejor esta variabilidad.

Por tanto, el margen de mejora no está solo en cambiar la arquitectura. También sería importante revisar las máscaras manuales, unificar criterios de anotación, aumentar ejemplos difíciles y probar aumentos de datos orientados a iluminación, encuadre y variabilidad anatómica. Además, sería conveniente usar un entorno con GPU correctamente configurada para poder entrenar durante más épocas y probar modelos de mayor capacidad.

8. Conclusiones

En esta PEC se han implementado y evaluado modelos de deep learning para localización, clasificación y segmentación. En la figura de Rey, las redes propias resuelven las tareas de forma aceptable, pero YOLO obtiene mejores resultados tanto en localización como en orientación.

En segmentación oral, la parte más importante ha sido construir un dataset utilizable combinando SMART-OM con imgsMIIA. Sobre ese dataset se entrenó una U-Net ligera y un YOLO segmentation. En la comparación final común, YOLO obtuvo mejores métricas globales, aunque U-Net fue superior sobre SMART-OM.

La conclusión principal es que la transferencia de aprendizaje resulta útil en datasets pequeños y heterogéneos. Sin embargo, los resultados también muestran que la calidad de las anotaciones y la homogeneidad del dataset son tan importantes como el modelo elegido. Como trabajo futuro, plantearía revisar los casos con Dice bajo, aumentar el número de imágenes anotadas, mejorar la homogeneidad de las máscaras y estudiar una segmentación multiclase para separar DT, VT y LT.

Referencias

- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*.
- Ultralytics. Documentación de YOLO para detección, clasificación y segmentación.
- SMART-OM dataset y anotaciones colaborativas imgsMIIA utilizadas durante la práctica.